CMP_SC 4050 Group Algorithm Engineering Assignment 2

From Brute Force to Better Algorithms: Divide and Conquer + Dynamic Programming

Connor Souders, Carson Mensink, Alexis Paten, Chris Crowder | CMP_SC 4050, Fall 2026

Code, CSV results, and reproduction commands are in the submission package (README.md). Results were produced with Python 3.14.3 on an Apple M3 (macOS, arm64), using only the standard library.

Part I. Divide and Conquer: Counting Inversions

An inversion in A[0..n−1] is a pair of indices i < j with A[i] > A[j].

Part 1A. The Two Algorithms

Brute-force pair counting

```
for i in range(n - 1):
    for j in range(i + 1, n):
        if arr[i] > arr[j]:
            count += 1
```

The inner loop runs n − 1 − i times for each i, so the comparison executes exactly (n − 1) + (n − 2) + … + 1 = n(n − 1)/2 times on every input. There is no early exit; only the number of increments (0 to n(n − 1)/2) depends on the order. Each iteration is constant work, so the running time is **Θ(n²)** in the best, average, and worst case, with Θ(1) extra space. Measured: doubling n from 1,024 to 8,192 multiplied the median time by 3.93, 3.99, and 4.04 (ideal 4).

Divide-and-conquer sort-and-count

sort_and_count(A) returns (inversion_count, sorted_list). If n ≤ 1 it returns (0, A). Otherwise it splits at mid = n // 2, recursively solves L = A[:mid] and R = A[mid:], and calls merge_and_count on the two sorted halves, returning $c_L + c_R + c_X$ and the merged list. The central step of the merge (1a/inversions.py):

```
if left[i] <= right[j]:
    merged.append(left[i]); i += 1
else:  # right[j] < left[i] <= left[i+1] <= ... <= left[-1]
    cross += len_left - i
    merged.append(right[j]); j += 1
```

Because the left half is sorted, a right element smaller than left[i] is smaller than all len_left − i remaining left elements, so those inversions are counted with one subtraction instead of one comparison each.

Correctness

For split point mid, every inversion (i, j) is exactly one of: a **left** inversion (i, j < mid), a **right** inversion (i, j ≥ mid), or a **cross** inversion (i < mid ≤ j). The classes are disjoint and cover all pairs, so the total is $c_L + c_R + c_X$ if each term is correct. By strong induction on n (the base case n ≤ 1 has no pairs), the recursive calls count exactly the inversions inside L and inside R, since each sees its elements in their original relative order.

A cross inversion is a pair $x \in L$, $y \in R$ with $x > y$. Every element of L precedes every element of R, so sorting each half does not change these pairs. When y is appended from the right, left[0..i−1] were already placed, which happens only when each was ≤ an earlier right element ≤ y, so none exceeds y; and left[i] > y with the left half sorted means all of left[i..] exceed y. So len_left − i is exactly the number of cross inversions involving y. If L runs out first, the remaining right elements add 0, which is correct because every x was placed before them. Each y is appended once and left elements never add to the count, so every cross inversion is counted exactly once. Ties take the left element, so equal values are never counted, matching the strict A[i] > A[j] definition. The merge is the standard mergesort merge, so the output is sorted(A).

Complexity

Slicing and merging each take Θ(n), so $T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + \Theta(n)$, $T(1) = \Theta(1)$. The recursion tree has $\lceil \log_2 n \rceil$ levels whose subproblems partition the n elements, so each level costs Θ(n); by the Master Theorem (a = b = 2, f(n) = Θ(n), case 2), **$T(n) = \Theta(n \log n)$** for every input order. Extra space is Θ(n) for the lists alive along one root-to-leaf path ($n + n/2 + \ldots \leq 2n$) plus Θ(log n) stack. An instrumented copy counted 8,948 merge comparisons at n = 1,024 ($0.87 \cdot n \log_2 n$) and 4,387,631 at n = 262,144 ($0.93 \cdot n \log_2 n$).

Part 1B. Validation and Runtime Scaling

Validation

The brute-force counter is the oracle. Every validation array must satisfy: (1) both counts agree exactly, (2) the returned list equals sorted(A), (3) neither function modified A, and (4) for hand cases, both equal the hand count (the inverted pairs are listed in validation_results.csv).

Table 1. Validation arrays.

Group	Count	Contents
Hand-checkable	15	empty, single, 2-element sorted/reversed, sorted, reverse sorted (n = 4, 5), nearly sorted (one swap, two swaps, one element moved), largest-first, textbook [2,4,1,3,5], random orders, negative values
Random distinct	300	n in 0–60, distinct values from −1000…999, seed 4050
Random with duplicates	50	n in 0–60, values 0–9; checks the strict > tie rule

All 365 passed. To confirm the suite can catch bugs, we ran two deliberately broken merges through it: replacing cross += len_left − i with cross += 1 failed 330 cases, and changing the tie rule <= to < failed 45, all in the duplicate group. Passing shows agreement on the tested inputs; the proof above covers all inputs.

Timing methodology

Inputs are seeded (2026) random permutations of 0..n−1, generated before timing; both algorithms get the same list at each n. Only the counter call is timed (timeit, garbage collector off); generation, CSV output, and plotting are outside the timed region, and plotting is a separate script. The loop count is calibrated so each repetition lasts at least 20 ms, each measurement is repeated 7 times, and the median per-call time is reported (min/max are kept in the CSV; spread was under 2% at n = 8,192). Sizes run from 2 to 4,096 for both algorithms, with fine steps near the crossover, then brute force to 8,192 and divide and conquer to 262,144.

Results

Table 2. Median seconds per call (1b/benchmark_results.csv). Bold: first size where divide and conquer is faster.

n	Brute force (s)	Divide & conquer (s)	Speedup (BF / D&C)
2	3.26e-7	1.21e-6	0.27×
16	6.41e-6	1.63e-5	0.39×
32	2.24e-5	3.64e-5	0.61×
48	5.61e-5	6.76e-5	0.83×
64	**9.53e-5**	**8.05e-5**	**1.18×**
128	3.77e-4	1.78e-4	2.11×
512	7.10e-3	8.68e-4	8.19×
1,024	2.67e-2	1.83e-3	14.6×
4,096	4.19e-1	8.17e-3	51.2×
8,192	1.69	1.73e-2	97.6×
262,144	—	0.720	—

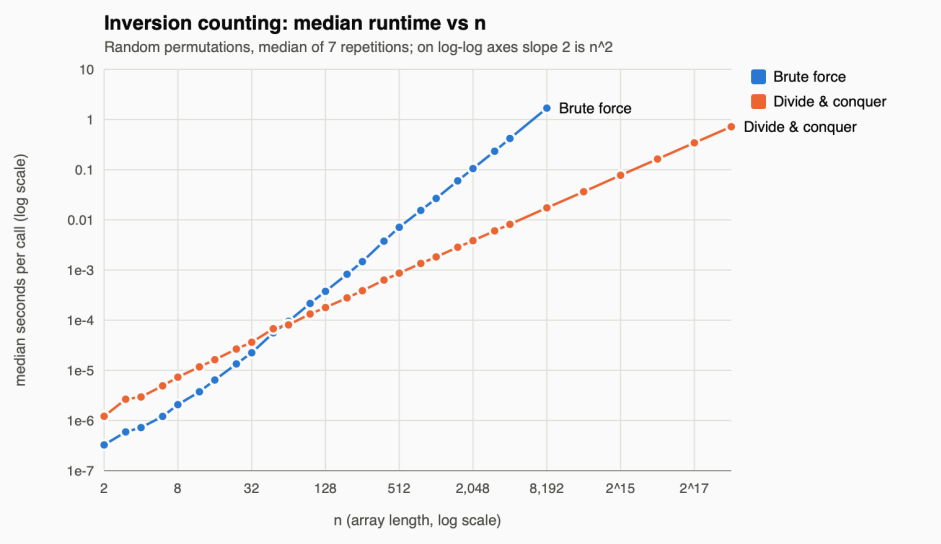


Figure 1. Median runtime vs. n on log-log axes. Brute force has slope ≈ 2; divide and conquer has slope just over 1. Speedup and normalized-runtime plots are in part-1/1b (speedup.svg, normalized.svg).

Interpretation

Growth matches the analysis. Doubling n multiplies brute-force time by ≈ 4 (3.93, 3.99, 4.04 from 1,024 to 8,192), as $\Theta(n^2)$ predicts, and divide-and-conquer time by 2.09–2.12 from 4,096 to 262,144, matching the $\Theta(n \log n)$ ratio $2 \cdot \log(2n)/\log n$ (2.17 at n = 4,096, 2.11 at 131,072). Normalized, t/n^2 for brute force levels off at about 25 ns for n ≥ 64, and $t/(n \log_2 n)$ for divide and conquer settles toward about 150 ns; flat curves confirm the growth terms. The speedup grows roughly like n / log n, from 1.18× at n = 64 to 97.6× at n = 8,192.

Crossover. On this machine divide and conquer becomes faster between n = 48 (0.83×) and n = 64 (1.18×) and stays faster at every larger size; below that, brute force wins by up to about 4×. The crossover is set by the constant factors that asymptotic notation drops: with times $c_{BF} \cdot n^2$ and $c_{DC} \cdot n \log_2 n$, they are equal where $n / \log_2 n = c_{DC} / c_{BF}$. At n = 64 we measured $c_{BF} \approx 23$ ns and $c_{DC} \approx 210$ ns, a ratio of about 9, which predicts n ≈ 50, consistent with the measured 48–64. Those constants depend on the **implementation** (each CPython recursive call, slice, and append allocates memory; a preallocated buffer or an insertion-sort cutoff would lower c_{DC}), the **language and interpreter version** (compiled code speeds up brute force's tight loop the most), and the **hardware** (cache size, memory bandwidth, branch prediction, and clock speed affect the two algorithms differently). So "48–64" describes this code on this machine, while Θ(n log n) beating Θ(n²) for large enough n holds everywhere.

Part II. Dynamic Programming: Coin Change

Part 2A. Four Coin-Change Solvers

All four solvers in part-2/2a/algorithms.py take (coins, amount) and return (coin_count, chosen_coins). Target 0 returns (0, []), and IMPOSSIBLE is returned as (−1, None). Negative targets and nonpositive denominations raise ValueError. Each solver sorts and deduplicates a copy of the denominations; the exact solvers try coins in ascending order and keep only strictly better counts, so results are deterministic.

A. Greedy (coin_change_greedy) scans from the largest denomination down, taking remaining // d copies of each coin d, which is the largest-feasible-coin rule applied in batches. Every returned coin is allowed and they sum to A, but the result need not be optimal, and (−1, None) from greedy only means it got stuck: for D = {3, 5}, A = 9 it takes 5 and cannot form 4, although 3+3+3 works.

B. Naive recursion (coin_change_naive) evaluates OPT(0) = 0 and OPT(a) = min{1 + OPT(a − d) : d ∈ D, d ≤ a}, where a minimum over no finite candidates is ∞. It caches no values; a write-only choice dictionary records a best coin for reconstruction but is never read to skip calls. An infinite OPT(A) returns (−1, None).

C. Top-down memoization (coin_change_memo) evaluates the same recurrence starting from memo[0] = 0, caches every computed amount (infeasible ones as ∞), and stores an optimal predecessor coin per feasible state for reconstruction. Only amounts reachable from A are visited.

D. Bottom-up DP (coin_change_dp) sets best[0] = 0 and all other entries to ∞, fills amounts 1..A in increasing order from best[a − d] + 1, and records each improving coin in last_coin[a]. Repeatedly subtracting the recorded coin from A reconstructs an optimal multiset.

Correctness of the exact solvers

By strong induction on a: the empty multiset is optimal for 0. For a > 0, any feasible solution contains some coin d, and the rest must be optimal for a − d (otherwise replacing it would improve the whole), so the optimum is among the candidates 1 + OPT(a − d); conversely, every finite candidate yields a feasible solution. So the minimum equals OPT(a), and no finite candidate means no solution. Naive recursion terminates because coins are positive; memoization returns the same values while reusing them; bottom-up is correct because every predecessor a − d < a is already solved. Each recorded coin satisfies OPT(a) = 1 + OPT(a − d), so reconstruction reaches 0 using exactly OPT(A) coins that sum to A.

Time and auxiliary-space analysis

Let n be the input list length, k the number of distinct denominations, c the smallest coin, $L = \lfloor A/c \rfloor$, and m the returned coin count. Preprocessing (validation, deduplication, sorting) costs $P = O(n + k \log k)$ time and $O(k)$ space. The returned list is excluded from auxiliary space.

Table 3. Complexity of the four solvers.

Solver	Time	Auxiliary space
Greedy	$O(P + k + m)$	$O(k + m)$ (temporary [coin] * take list)
Naive recursion	$O(P + N(A))$, where $N(a) = 1 + \Sigma_{d \le a} N(a - d)$; at most $O(k^{L+1})$, exponential in A when $k \ge 2$	$O(k + A)$: up to $L + 1$ stack frames plus the choice dictionary
Top-down memo	$O(P + kA)$: each reached state is computed once and checks ≤ k coins	$O(k + \lvert S \rvert)$ for the reached amounts S, $\lvert S \rvert \le A + 1$ (cache, choices, stack)
Bottom-up DP	$O(P + kA)$: $A + 1$ states, ≤ k coins each	$O(k + A)$

$O(kA)$ is pseudo-polynomial because A needs only $\lfloor \log_2 A \rfloor + 1$ bits (see Part 2C). The recursive solvers can also exceed Python's recursion limit for large A/c; bottom-up DP avoids that.

Part 2B. Exact-Solver Validation and Greedy Counterexamples

Validation

Before DP is used as ground truth, part-2/2b/main.py cross-validates the three exact solvers. An independent validator checks every returned list: its coins sum to the target, its length equals the reported count, and every coin is a denomination; failures must be exactly (−1, None). Different lists are accepted when the optimal counts agree.

The 12 hand-checkable cases cover target 0 with and without denomination 1, impossible targets, single-denomination systems, greedy traps, and multiple optima (D = {2, 3, 4}, A = 6: both [3, 3] and [2, 4]); each is checked against a hand-counted answer, which also checks the impossibility results. The 150 random cases (seed 4050) use 2–5 denominations always including 1, others sampled from 2–14, and targets 0–22 so naive recursion stays practical. **All 162 cases passed** for all three solvers.

Search method and results

The search examines all 91 systems $[1, c_2, c_3]$ with $1 < c_2 < c_3 \le 15$ and targets 1–25 (2,275 instances), running greedy and bottom-up DP and validating both lists. A case is recorded only when greedy is feasible but uses more coins than DP. It found **197 counterexamples**; greedy_counterexamples.csv lists the required sanity case first, then the next two distinct systems in deterministic search order (Table 4).

Table 4. Three greedy counterexamples.

Denominations	Target	Greedy (count)	Optimal (count)	Extra coins
{1, 3, 4}	6	4 + 1 + 1 (3)	3 + 3 (2)	1
{1, 4, 5}	8	5 + 1 + 1 + 1 (4)	4 + 4 (2)	2
{1, 4, 6}	8	6 + 1 + 1 (3)	4 + 4 (2)	1

Why greedy fails

For D = {1, 3, 4} and A = 6, greedy commits to 4, leaving 2, which only 1s can fill: [4, 1, 1], three coins. Taking 3 first leaves another 3, giving [3, 3]; one coin cannot make 6, so two coins is optimal. The locally largest coin leaves a remainder that the middle denomination can no longer complete, so the better global combination is unreachable once 4 is chosen. The other cases fail the same way: 5 or 6 leaves a remainder needing 1s, while two 4s make 8.

Part 2C. Memoization and Pseudo-Polynomial Behavior

Before the experiments, part-2/2c/part2c_experiments.py cross-checks the three exact solvers on 239 more cases (39 fixed cases with targets 0–12 for {1, 3, 4}, {1, 4, 6}, and {4, 6}, plus 200 random cases with seed 42), checking each bottom-up list's count, sum, and coins. All passed.

Experiment 1: Repeated subproblems

With D = {1, 3, 4}, instrumented copies of the naive and memoized recurrences (same ascending coin order) run for A = 5, 10, 15, …, counting recursive calls, distinct amounts reached, and cache hits. The memo starts with 0 cached, so reaching 0 counts as a hit. Naive recursion stops at a documented safety cutoff of 250,000 calls. Completed answers are asserted equal to the Part 2A solvers, and all counts are deterministic.

Table 5. Call counts (part2c_experiment1.csv).

Target A	Naive calls	Naive distinct states	Memo calls	Memo states	Cache hits
5	15	6	11	6	6
10	168	11	26	11	16
15	1,869	16	41	16	26
20	20,736	21	56	21	36
25	229,970	26	71	26	46
30	250,000 (stopped)	31	86	31	56

Both solvers evaluate the same recurrence, OPT(a) = min{1 + OPT(a − d)}; the only difference is whether a solved amount is remembered. Naive recursion re-expands an amount every time a different coin sequence reaches it, so its calls grow about 11× per +5 in A: at A = 25 it made 229,970 calls for only 26 distinct amounts. Memoization made 71 calls, 46 of them cache hits, computing each of the 25 positive states once. At A = 30 naive recursion hit the cutoff (250,000 is only a lower bound), while memoization finished in 86 calls. Memoized calls grow by 3 per unit of A (one per coin), consistent with O(kA).

Experiment 2: Pseudo-polynomial scaling

Keeping D = {1, 3, 4}, we doubled A from 2^8 to 2^{18} and timed the real coin_change_dp seven times per target with time.perf_counter(), reporting the median. Timing includes preprocessing and reconstruction but not validation or file output. The DP table has A + 1 states.

Table 6. Bottom-up DP scaling (part2c_experiment2.csv).

Target A	Bits in A	DP states	Median runtime (ms)
256	9	257	0.0887
512	10	513	0.1896
1,024	11	1,025	0.3925
2,048	12	2,049	0.8245
4,096	13	4,097	1.7005
8,192	14	8,193	3.4363
16,384	15	16,385	7.0112
32,768	16	32,769	13.8394
65,536	17	65,537	27.8256
131,072	18	131,073	55.0530
262,144	19	262,145	110.6298

Runtime doubles with A (the last three doublings: 2.01×, 1.98×, 2.01×), the linear growth that O(kA) predicts for fixed k. But each doubling adds only one bit to A. With $b = \lfloor \log_2 A \rfloor + 1$ bits, A can be as large as $2^b - 1$, and here $A = 2^{b-1}$, so the states and runtime grow exponentially in b: going from 9 to 19 bits multiplied runtime by about 1,250. O(kA) is therefore **pseudo-polynomial**: polynomial in the numeric value of A but not in the length of its binary encoding. Memoization removes repeated states but not this dependence, because with coin 1 every amount from 0 to A is reached.

Part III. Cross-Paradigm Engineering Reflection

1. What work does divide and conquer avoid compared with checking every pair?

Brute force compares all n(n − 1)/2 pairs. Divide and conquer never compares cross pairs one at a time: when right[j] < left[i], the merge counts all len_left − i remaining left elements in one step, because sorting has already ordered them. Each level does O(n) work over O(log n) levels, so $\Theta(n^2)$ becomes Θ(n log n); in our benchmark that was 97.6× faster at n = 8,192.

2. Why are inversion subproblems independent, while coin-change subproblems overlap?

Inversion subproblems are disjoint index ranges: each half counts its own inversions without the other half's answer, and the merge handles only the crossing pairs, so no subproblem is ever solved twice. Coin-change subproblems are remaining amounts, and many choice sequences reach the same amount: with {1, 3, 4} and target 6, taking 4 leaves 2, and taking 1 then 3 also leaves 2. Both need OPT(2), which is why naive recursion made 229,970 calls for 26 distinct amounts at A = 25.

3. What role does sorting play in the divide-and-conquer improvement?

Sorted halves make cross inversions countable in batches: if right[j] < left[i], it is smaller than every later left element, so the merge adds len_left − i at once, and taking the left element on ties keeps equal values uncounted. Sortig the whole array first would destroy the order being measured; instead each half is sorted only after its internal inversions are counted, which is safe because every left position precedes every right position.

4. Why can naive recursion and memoization have dramatically different runtimes?

Both evaluate the same recurrence, but naive recursion rebuilds an entire subtree each time an amount recurs, while memoization computes each of at most A + 1 amounts once with at most k candidates, giving O(kA) instead of exponential time. At A = 30 naive recursion hit the 250,000-call cutoff while memoization needed 86 calls. The cost is O(A) space, and the pseudo-polynomial dependence on A remains.

5. How did baseline and exact implementations help validate the faster algorithms?

Brute force follows the inversion definition directly, so it was the oracle for divide and conquer on 365 arrays, including duplicates that catch a wrong tie rule. For coin change, three differently structured exact solvers agreed on 162 Part 2B cases and 239 Part 2C cases, with an independent validator checking sums, counts, and coins. The validated DP then exposed 197 greedy failures (Table 4). Agreement is evidence only on tested inputs, since shared bugs could make implementations agree on a wrong answer, so the correctness proofs are still needed.

6. When would you try divide and conquer versus dynamic programming?

We would try divide and conquer first when a problem splits into independent, non-overlapping parts whose answers combine cheaply, as the linear merge does for inversions; caching those calls would gain nothing because no part repeats. We would try dynamic programming when different choice sequences lead to the same states and an optimal answer is built from optimal answers to smaller states, as with remaining amounts in coin change. Either way, we would first estimate the number of subproblems: an O(kA) DP is impractical for a huge binary-encoded A.

Contribution Statement

Connor Souders: Part I (1A, 1B): inversion counters, analysis, validation, benchmarks, and graphs.
Carson Mensink: Part 2A solvers and complexity analysis; final Part 2B validation pipeline.
Alexis Paten: Part 2B validation and greedy counterexample search; repository setup.
Chris Crowder: Part 2C experiments; Part III reflection.

AI Use Statement

The group used Claude (Anthropic), including Claude Code, for coding assistance, debugging, test generation, and documentation in all parts of this assignment. AI output was not treated as evidence of correctness. Each optimized algorithm was checked against an independently implemented baseline or exact solver: the divide-and-conquer counter against brute force on 365 arrays, and the three exact coin-change solvers against each other and an independent solution validator. Before submission, the validation suites and Part 2C experiments were rerun from a clean copy of the submission.